

Document Number: P0978R0

Date: 2018-03-31

Audience: Evolution

Revises: none

Reply to: Gor Nishanov (gorn@microsoft.com)

A Response to "P0973r0: Coroutines TS Use Cases and Design Issues"

Introduction

A coroutine is a generalization of a function that in addition to usual control flow operations such as call and return, can suspend execution of itself and yield control back to the caller with an ability to resume execution at a later time. In C++, coroutines were explicitly designed to efficiently and succinctly support the following use patterns:

- asynchronous tasks, where `co_await <expr>` suspends a coroutine while waiting for the result of an expression) and the coroutine is resumed once the result is available;
- generators, where `co_yield <expr>` suspends the coroutine yielding the result of the expression to the consumer and the coroutine is resumed once the consumer asks for the next value;
- asynchronous streams, which can be thought of as an asynchronous version of a generator where both `co_await` and `co_yield` can be used.

Unlike most other languages that support coroutines, C++ coroutines are open and not tied to any particular runtime or generator type and allow libraries to imbue coroutines with meaning, whereas the compiler is responsible solely for efficient transformation of a function to a state machine that is the foundation of the coroutine.

Because C++ coroutines are open in nature and semantic provided by the library, they can be applied to some non-traditional use cases, such as automatic error propagation of `expected<T, E>`, which happened to be a major use case in Google.

P0973r0 paper identified a number of issues that make coroutines in their current form sub-optimal for exception-less error propagation. We agree with some of the issues (for example, awkwardness of `co_return` and `co_await` keywords in that scenario), but we categorically, absolutely, emphatically, vociferously object to notion that coroutines violate zero-overhead principle. Before diving into details of why we believe that P0973r0 is mistaken on this issue, let's go through the areas of agreement (or mostly agreement).

Const reference parameters are dangerous

We agree with P0973 point here and would go even further. Any reference or raw pointer is dangerous if that reference/pointer survives and used after the lifetime of an object has ended.

While it is a hazard in C++ in general. It is a likely hazard in asynchronous scenarios. The mitigation is available in coroutine design itself.

Though standard coroutine types are unlikely to be that drastic, custom coroutine types used in a particular codebase by a particular company can chose to ban reference and pointer arguments to a coroutine with an escape hatch with some version of `std::ref` wrapper, for example, when a developer is sure that references use is safe. It only requires a little bit of template meta-programming and a `static_assert` when defining a coroutine type.

Note that the banning is done by the library defining the semantics of the coroutine. It will be a compile time error to declare a parameter that is deemed unsafe by the coroutine type designer. No coding guidelines or static analyzers are required. The code won't compile if a parameter of unacceptable type is declared in a coroutine.

Banning return is user-hostile, and make migration difficult

We agree with authors' points. There is no technical reason for why coroutines cannot use `return` in place of `co_return`.

If there is a will of the committee, we can revisit the issue and explore the alternatives that can address this concern. For example:

- `return` / `co_return` are interchangeable in coroutines
- `co_return` is gone completely (warning: breaking change)
- `return` can only be used in *"non-suspending"* coroutines, such as the ones used with `expected<T, E>`, whereas traditional coroutines will still be required to use `co_return`.

The name `co_await` privileges a single use case

Indeed, the names of keywords `co_await` and `co_yield` bake in certain expectations of what semantics library should provide with `co_await` implying waiting for some value to get into the coroutine and `co_yield` implying pushing some value out of the coroutine.

Even though `co_yield` can be implemented purely in terms of `co_await` we chose to have dedicated `co_yield` keyword, in order to anticipate enabling return type deduction in coroutines N4499/[dcl.spec.auto]/16. These keywords used alone or together nicely cover the intended design space:

```
// lambda return type deduces to generator<int> (C++20)
[] { for (int i = 0; i < 10; ++i) co_yield i; }

// function return type deduces to task<double> (C++20)
auto f() { co_await foo(); co_return 3.14; }
```

```
// lambda return type deduces to async_stream<size_t> (post C++20)
[] { for (;;) {
    size_t val = co_await read_async();
    co_yield val;
}
};
```

Replacing meaningful `await` and `yield` keywords with some semantic-less keyword or symbol may make coroutines less readable, less user-friendly and is likely to make automatic type deduction in coroutines difficult if not impossible.

constexpr is not supported

We intentionally kept the scope of coroutine design *relatively* small but sufficient to cover the design space with the expectation that as major compiler vendors implement the feature, become familiar with it, gain better understanding of related issues, we will be better equipped to evolve the coroutines in the future. `constexpr` coroutines was one of the things that was cut to keep the design and implementation manageable.

While we do not want to rush `constexpr` in coroutines in general, we could consider `constexpr` for *non-suspending* coroutines if that is critical for Google use cases.

In earlier discussions with Richard Smith, we discussed how we can make it easier on the compiler to deal with cases like `expected<T, E>` where no actual suspension and resumption happens by allowing library writer to indicate that to the compiler. For an example, as a strawman, by specializing a trait:

```
template <typename T, typename E>
struct is_non_suspending<expected<T,E>>: true_type {};
```

co_await chains awkwardly

We believe that precedence for `co_await` allows excellent composition properties for intended usage scenarios.

When using awaitable composition prior to applying operator `co_await` as it is the case with altering executor for resuming or requesting a different error handling mode:

```
int v = co_await s.async_read().on_executor(e);
expected<v, error_code> v = co_await s.async_read().as_expected_ec();
```

when sending the result of the `co_await` to standard algorithms via view composition:

```
int v = co_await when_all(s1.async_read(), s2.async_read())
    | reduce_exceptions()
    | accumulate(0);
```

and even when combining all of them together in one expression:

```
int v = co_await when_all(s1.async_read(), s2.async_read())
    .on_executor(e)
    | reduce_exceptions()
    | accumulate(0);
```

Coming back to the issue of using parentheses when mixing postfix and prefix operators raised in P0973r0:

Yes. Mixing postfix and prefix operators require wrapping an expression in parentheses or splitting it out into a separate line.

```
int size = (co_await GetString(some_complicated_expression)).size();

// or

const auto str = co_await GetString(some_complicated_expression);
// use str.size() or assign it to a variable
```

Giving that `co_await` currently cleanly composes with `await` transformers and view composition operators, having to add sometimes extra parentheses or split out an *await-expression* for clarity seems like a non-issue.

Moreover, in the particular case shown by the authors, it is unlikely that the *awaitable* returned from `GetString` would have anything but `await_ready`, `await_suspend`, and `await_resume` members defined, so an attempt to mistakenly extract `size()` from the *awaitable* will be immediately caught by the compiler with a helpful *Fix-it* explaining how to fix it.

The library bindings are massive and yet inflexible

While adjectives characterizing flexibility and girth of coroutine library bindings are subjective, the fact is that Coroutine TS carefully balanced library binding in favor of readability and ease of use over the beauty of the core wording.

Coroutine bindings are usually only a few lines of code and easy to specify. In fact, they are so compact that Coroutine TS shows entire generator implementation as an example.

Let consider other points raised by the authors:

the library extension points are ... keyed off of the function signature ... This effectively disallows per-function customization.

This is correct, per function customization was intentionally cut from the design to keep it small and that decision was proven to be correct. In four years and with thousands of users there was not a single customer asking for per function customization.

A particular form of per function customization that was considered and cut was:

```
task<T> foo() using(different-coroutine-trait-than-default-one) {  
    ...  
}
```

where *using-traits-clause* would appear only in function definition.

implicit core/library coupling inside `coroutine_handle` will mean that many kinds of implementation changes will require NxM coordination between vendors of compilers and standard libraries.

If one looks at standard library headers of their favorite library vendor, one will discover that many library facilities are implemented using compiler intrinsics, `__builtin_launder`, `__builtin_addressof`, `__builtin_nan`, `__is_abstract`, `is_union`, `__is_class` or `__is_final`, to name a few. Adding 3-4 more intrinsics to the list of a hundred, does not make existing situation substantially different. MSVC, clang and GCC do attempt to harmonize their intrinsics to be able to compile each other standard libraries. The `<coroutine>` header is no different.

Unlike the actual coroutine transformation where implementation may vary dramatically between vendors, `coroutine_handle` type in question is a tiny wrapper around a `void*` and there is not much room to go wild. At the moment MSVC and Clang differ in only one intrinsic, but, we (MSVC) plan to match Clang before C++20.

Member	libcxx	MSVC
resume()	<code>__builtin_coro_resume</code>	<code>_coro_resume</code>
destroy()	<code>__builtin_coro_destroy</code>	<code>_coro_destroy</code>
done()	<code>__builtin_coro_done</code>	<code>_coro_done</code>
promise()	<code>__builtin_coro_promise</code>	N/A

Now let's proceed to the most controversial claim of P0973r0 with which we disagree violently and emphatically!

Implicit allocation violates the zero-overhead principle

Brief recap of rationale for the C++ Coroutine design

The design of C++ coroutines was a delicate balancing act weighting different concerns against each other that led to a decision to rely on elidable implicit memory allocation for the coroutine frame by default and giving a coroutine designer an option to override the default allocation if desired.

We prioritized having zero-overhead coroutines with light-weight syntax out of the box over losing some control over allocation of the coroutine frame to a compiler. Note that in the regular functions developers have zero control of how activation frames are allocated.

Zero-overhead out of the box, meant that we wanted an experience where developers do not have to pack their entire logic into a single coroutine out of fear that breaking it out into smaller coroutines for readability and good hygiene will impose overhead. We also categorically did not want to force them to deal with custom allocators for simple tasks like breaking one coroutine into smaller pieces. The following has to work efficiently out of the box without requiring users to tinker with custom allocators and without restrictions on how many nested coroutines could be called in this fashion (without recursion).

```
task<> big_task() { // lifetime of `subtask` is fully enclosed in its caller
    ...
    co_await subtask(); // coroutine frame allocation elided
    ...
}
```

We also wanted to have generators that are as efficient as any other way of expressing a range of lazily produced values:

```
generator<int> range(int from, int to) {
    for (int i = from; i < to; ++i)
        co_yield i;
}
int main() { // lifetime of `range` is fully enclosed in its caller
    auto s = range(1, 10); // coroutine frame allocation elided
    return std::accumulate(s.begin(), s.end(), 0);
}
```

The requirements for this optimization to occur is that your coroutine type has to have RAI semantics, some members of the coroutine type (i.e. generator or task) should be available for inlining and the lifetime of the coroutine does not escape its caller (see P0981r0 for more detailed exposition). Here are the examples where there **will be** an allocation of memory (and coroutine designer can control what allocator is used if desired).

```
task<> session(tcp::socket s, size_t block_size) { ... }

task<> server(io_context& io, tcp::endpoint const& endpoint, size_t block_size)
    tcp::acceptor acceptor(io, endpoint);
    acceptor.listen();
    for (;;) // coroutine `session` escapes `server`. Will require allocation
```

```

        spawn(io, session(co_await async_accept(acceptor), block_size));
    }

```

Similarly, if you start moving generators around, they will need allocation:

```

main() { // coroutine `seq` escapes its caller
    auto gen = seq();
    // do something with gen
    do_more_work(move(gen)); // escapes, `seq` will require allocation
}

```

Now that we gave a brief overview of the rationale behind the decision to rely on implicit elidable allocation of the coroutine frame, let's proceed to the concerns expressed in P0973r0.

Concerns about implicit elidable coroutine frame allocation

Let me briefly restate authors points in the condensed form (please see the original paper [P0973r0] for full context):

- not clear it is feasible in general, clang for example only do it when the coroutine is available in the same translation unit
- even if eventually this optimization will be reliable, programmers won't trust it
- at odds with design philosophy of C++
- overloading of operator new is obscure and not flexible enough
- and besides, allocation is not needed at all for expected<T,E>

Let's start with analyzing the first point:

- not clear it is feasible in general, clang for example only do it when the coroutine is available in the same translation unit

Indeed, current implementation of coroutines in clang only runs heap elision optimization if coroutine is defined in the same translation unit as its user. However, it is purely a temporary situation and requires a little bit more investment in the compiler to remove the limitation.

Moreover, Richard Smith, Chandler Carruth and Gor Nishanov, on one rainy evening of 2014, sketched out how this optimization would work across hard ABI boundaries where peeking at the body of the coroutine is impossible.

With respect to feasibility of this optimization in general, if there is a **reasonable doubt** that this optimization is not feasible for the designed use cases, we need to absolutely stop work on the Coroutines TS and look for alternatives! **Update:** At the end of the Jacksonville 2018 meeting, Google and Microsoft compiler engineers have met and written a join statement on feasibility of heap allocation elision optimizations (see: P0981R0: "Halo: Coroutine Heap Allocation eLision Optimization: the joint response") where feasibility of this optimization was reaffirmed.

Elidable implicit allocation of coroutine frame is the foundational point in the design. This is what gives the coroutine light-weight syntax combined with zero-overhead. If it does not work for the targeted use cases, we need to rethink our approach to the coroutines. So far, Clang and MSVC implementors are in agreement about feasibility. GCC have not tried implementing coroutines yet, but given that Clang and MSVC think that they can do it, GCC is likely could as well.

- even if eventually this optimization will be reliable, programmers won't trust it

Authors present hesitance of returning large object by value due to distrust in copy elision as an analogy why developer may distrust implicit elidable frame allocation. This analogy does not work with coroutines.

With copy elision, developers have relatively straightforward alternative to returning by value, namely, adding a reference parameter. With coroutines, there is no easy alternative.

Coroutines address such a dire need, that developers are grabbing raw compiler bits of incomplete non-yet-standard feature and start using it, alternatives to coroutines are significantly more verbose and/or unreadable/maintainable. It seems that this matches the view of the authors of P0973r0 as well who concur in the introduction that coroutines address such a dire need that their developers will be grabbing non-standardized-bits as well: "we believe that coroutines have the potential to solve several problems for C++ programmers at Google, and moreover those problems are likely serious enough, and the potential solution good enough, to justify the the risk of adopting coroutines prior to full standardization."

Unlike copy-elision where there is an easy alternative, coroutines are so badly needed that concern that coroutines will not be used because of mistrust in the compiler is much less likely.

- at odds with design philosophy of C++

Please see the earlier section: "Brief recap of rationale for the C++ Coroutine design" where philosophy behind coroutine design was explained and, in authors view, is fully in line with design philosophy of C++. This view is shared by many experts including the creator of C++ Bjarne Stroustrup.

- overloading of operator new is obscure and not flexible enough

As a core language feature, coroutines rely on core language facilities, such as overloading of `operator new` to control allocations. On the library side, coroutine designers, working on user-facing library types can chose to expose customization points for allocation in traditional ways of the libraries, namely, with full richness of the `std::allocator` and its friends.

In the draft version of P0973r0 authors claimed that it is impossible to store coroutines with small state on the stack and with larger state on the heap, in the latest revision, authors state that Coroutines TS does not give tools to coroutine designer to allow stack-like allocation for nested generators.

While we understand that authors have these concerns, we are happy to report that in both cases, the limiting factor is not the coroutine TS customization points, but the lack of imagination on behalf of the coroutine designer. Both are possible with rather straight-forward code.

- and besides, allocation is not needed at all for expected `<T,E>`

We are in full agreement with authors of P0973 on this subject. **Moreover**, efficient use of coroutines with `expected<T, E>` **absolutely, does not rely on the heap elision optimization at all!** We only rely on Coroutine TS blessing not to have an allocation if not needed.

In llvm, there is a very simple coroutine optimization called "suspend point simplification and elimination", which looks to see if it can simplify and get rid of suspend points. Suspend point simplification looks for cases where an expansion of suspend point would lead to pure local control flow (continue execution or jump to the coroutine end) and if it is, a suspend point is removed and replaced with a normal local control flow within a function. If **all** suspend points are eliminated from the coroutine due to simplification or unreachability, all of the coroutine-ness is stripped out and the coroutine becomes a normal function. No allocations, no coroutine transformations are required.

As mentioned earlier, when discussing `constexpr`, we don't have to make the optimizer work that hard to turn a coroutine back to a function if we can explain to the compiler that coroutines using `expected<T, E>` are not coroutines at all and can be completely dealt with by the frontend of the compiler.

Now, while it is flattering that coroutines are so flexible and efficient that they can be applied to things which are not coroutines at all, this discussion makes one think that maybe using coroutines for expansion of `expected<T, E>` is not necessarily a good thing for C++ in the long term. Maybe, working on making exceptions more acceptable to developers and finding ways to evolve exceptions to deal with cases where today `expected<T, E>` is preferable could be a more rewarding long-term goal.

Conclusion

We thanks the authors of P0973r0 for taking time to try out coroutines and write a concerns paper. We are in agreement with some of the concerns, believe that some are not relevant to Coroutines TS as a solution is available via library bindings, and other concerns can be addressed with small non-breaking improvements, if desired, to improve support for non-exceptional error propagation via `expected<T, E>`.

As we understand P0953r0 paper was written partially to defend and contrast an alternative coroutine design to be presented in Rapperswil. Without seeing the alternative design, it is difficult to evaluate whether it addresses the concerns fully without introducing its own set of problem. This document provides responses to reported issues. While we provide a vigorous defense of the current TS, we do not preclude a possibility that some of the ideas from not-yet-revealed alternative design cannot be beneficially added to the existing TS to address some of the expressed concerns.

Acknowledgements

Thanks to Bjarne Stroustrup, Geoffrey Rommer, Casey Carter, and many others, for feedback on previous drafts of this paper.

References:

- P0973R0: "Coroutines TS Use Cases and Design Issues" shared on reflector (<https://wg21.link/P0973r0>)
- N4499: <http://open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4499.pdf>

- Coroutines TS: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4723.pdf>
- Coroutines using allocator example: <https://godbolt.org/g/QjRTet>
- Coroutines storing themselves in `array<byte, N>`: <https://godbolt.org/g/PiKYg2>.
- P0981R0: "Halo: Coroutine Heap Allocation eLision Optimization: the joint response"
(<https://wg21.link/P0981r0>)